



Universidad
Tecnológica
de Pereira

Facultad de Ingenierías
Ingeniería de Sistemas y Computación
IS453

ProgIIIIG1

Actividad02 : Backtracking y SLD

Andrés David Rincón Salazar
andres.rincon2@utp.edu.co
ID: 11.289.05.513

Pereira, Colombia
- Octubre 2026 -

Índice general

1. Introducción	2
2. Punto 1: Puentes de Königsberg	4
2.1. Árbol SLD	7
3. Punto 2: Rutas más cortas	9
3.1. Árbol SLD	11
4. Punto 3: Grafo dirigido	14
4.1. Árboles SLD	17
4.1.1. Versión segura (árbol completo)	17
4.1.2. Versión ingenua (árbol infinito)	18
5. Repositorio	21

Capítulo 1

Introducción

Esta actividad presenta tres programas en lenguaje Prolog sobre grafos, en los que la respuesta se obtiene por *backtracking* y por resolución *SLD*:

- **Punto 1:** problema de los puentes de Königsberg. Las cuatro masas de tierra son los vértices de un multigrafo y los siete puentes son sus aristas. Se determina si existe un paseo que cruce cada puente una sola vez y regrese al punto de partida, mediante el teorema de Euler y mediante búsqueda exhaustiva con backtracking.
- **Punto 2:** rutas más cortas sobre el mapa de carreteras de España (grafo no dirigido con distancias en km). Los caminos se enumeran por backtracking con una lista de ciudades visitadas y la ruta más corta es el mínimo entre todos ellos.
- **Punto 3:** grafo dirigido de ciudades de Canadá (Vancouver, Edmonton, Calgary, Saskatoon, Regina y Winnipeg). Responde las preguntas de la presentación, incluye reglas para saber si un nodo tiene aristas y para el costo de ir de X a Z pasando por Y, y contrasta una versión ingenua de la alcanzabilidad (que entra en bucle en el árbol SLD) con una versión segura.

Cada programa incluye su encabezado descriptivo (propósito, predicados y salvedades), comentarios de sección sobre hechos y reglas, y un bloque de consultas de validación con sus resultados.

Al final de cada punto se incluye el árbol SLD de su consulta principal. Los árboles se generaron con el paquete *SLDNF Draw* de SWI-Prolog (`pack_install(sldnfdraw)`) a partir de las mismas cláusulas del programa, y se dibujaron con el paquete `forest` de L^AT_EX. Los archivos de entrada y el script que los regenera están en la carpeta `arboles/fuentes`. Para que la herramienta pueda dibujarlos, en esos archivos `\+` se escribe `not/1` (es la misma negación por fallo) y las variables anónimas `_` llevan nombre. La notación de los árboles es:

- Cada nodo es el resolvente (los objetivos pendientes); las variables llevan un subíndice que indica en qué paso de resolución fueron renombradas.

- El texto azul entre llaves es la ligadura (sustitución) que produjo la unificación con la cláusula elegida.
- Un recuadro es una negación por fallo: dentro está el subárbol de `memberchk/2` y su resultado. Si `memberchk` resulta `true`, la negación falla y la rama termina en `false`.
- `true` (verde) es una rama de éxito, `false` (rojo) una rama fallida y $\dots$ un subárbol que existe pero no se dibuja.
- Prolog recorre el árbol en profundidad y de izquierda a derecha; el *backtracking* es el paso de una rama a la siguiente rama hermana a la derecha.

Los árboles son anchos; el PDF es vectorial, así que se pueden ampliar sin perder nitidez.

Capítulo 2

Punto 1: Puentes de Königsberg

Listing 2.1: ProgIIIG1-Act02-Pto01-Andres_David_Rincon_Salazar.pl

```
/*
=====
Publication Date: 2026-10-02
Publication Time: 23:45
Code Version: 1.0
Author: Ing(c) Andres David Rincon Salazar
Programming Language: Prolog
Language Version: ISO/IEC 13211-1 (SWI-Prolog 10.0.2)
Compiler: SWI-Prolog 10.0.2
Operating System: Windows 11 - 24H2
Presented to: Ramiro Andres Barrios Valencia
University: Universidad Tecnologica de Pereira
Program: Ingenieria de Sistemas y Computacion

Program Description:
Koenigsberg bridges problem. The four land masses are the vertices of a
multigraph and the seven bridges are its edges. The question is whether a
walk exists that crosses every bridge exactly once and returns to the
starting point (Euler circuit). It is answered in two ways:
  1. Euler's theorem: a connected multigraph has an Euler circuit if and
    only if every vertex has even degree.
  2. Exhaustive search by backtracking: the program tries every walk and
    fails when none closes using all the bridges.
Validation queries are included at the end of the file.

Predicates description:
- zona(Zona): (fact) land mass (vertex).
- puente(Id, A, B): (fact) bridge Id joins land masses A and B.
- cruce(Id, Desde, Hasta): (rule) bridge crossed in either direction.
- grado(Zona, Grado): (rule) number of bridges touching Zona.
- grado_par(Zona): (rule) Zona has even degree.
- circuito_euler_teorema: (rule) every zone has even degree.
- paseo(Inicio, Pasos): (rule) closed walk from Inicio using each bridge
  exactly once, found by backtracking. Pasos = [paso(Id, De, A), ...].
```

- hay_paseo: (rule) some starting zone admits such a walk.

Warnings (Salvedades):

- The slide gives no figure, so the bridge layout is taken from the historical map: norte and sur are the river banks, kneiphof and este (Lomse) are the two islands; 2 bridges norte-kneiphof, 2 sur-kneiphof, and 1 each for norte-este, sur-este and kneiphof-este.
- The two parallel bridges between norte-kneiphof and sur-kneiphof are distinct facts (b1/b2 and b3/b4); that is why the graph is a multigraph.
- circuito_euler_teorema/0 only checks the degrees; connectivity of the graph is assumed (it holds for Koenigsberg), not verified by the program.
- paseo/2 has no solution for this graph; the search is exhaustive but small (7 bridges), so it ends quickly.

*/

```

=====
%=====
% FACTS: land masses (vertices)
%=====
zona(norte).
zona(sur).
zona(kneiphof).
zona(este).

%=====
% FACTS: the seven bridges (edges), puente(Id, ZonaA, ZonaB)
%=====
puente(b1, norte, kneiphof).
puente(b2, norte, kneiphof).
puente(b3, sur, kneiphof).
puente(b4, sur, kneiphof).
puente(b5, norte, este).
puente(b6, sur, este).
puente(b7, kneiphof, este).

%=====
% RULE: a bridge can be crossed in both directions
%=====
cruce(Id, Desde, Hasta) :- puente(Id, Desde, Hasta).
cruce(Id, Desde, Hasta) :- puente(Id, Hasta, Desde).

%=====
% RULES: degree of a vertex and Euler's theorem
%=====
grado(Zona, Grado) :-
    zona(Zona),
    findall(Id, cruce(Id, Zona, _), Puentes),
    length(Puentes, Grado).

grado_par(Zona) :-
    grado(Zona, Grado),
    0 is Grado mod 2.

```

```

% The graph is connected, so the theorem reduces to "all degrees are even".
circuito_euler_teorema :-
    forall(zona(Zona), grado_par(Zona)).

%=====
% RULES: exhaustive search by backtracking
% caminar/5 crosses an unused bridge at each step; when it chooses a dead
% end, Prolog backtracks and tries the next bridge.
%=====
paseo(Inicio, Pasos) :-
    zona(Inicio),
    aggregate_all(count, puente(_, _, _), Total),
    caminar(Inicio, Inicio, Total, [], PasosInv),
    reverse(PasosInv, Pasos).

% Base case: back at the start with every bridge used.
caminar(Fin, Fin, Total, Usados, Usados) :-
    length(Usados, Total).
caminar(Actual, Fin, Total, Usados, Resultado) :-
    cruce(Id, Actual, Siguiente),
    \+ memberchk(paso(Id, _, _), Usados),
    caminar(Siguiente, Fin, Total, [paso(Id, Actual, Siguiente)|Usados], Resultado).

hay_paseo :-
    paseo(_, _), !.

/*
=====
Validation queries
=====
?- grado(norte, G).
G = 3.
?- grado(sur, G).
G = 3.
?- grado(kneiphof, G).
G = 5.
?- grado(este, G).
G = 3.
?- grado_par(kneiphof).
false.
?- circuito_euler_teorema.
false.
?- paseo(norte, Pasos).
false.
?- paseo(kneiphof, Pasos).
false.
?- hay_paseo.
false.
=====
*/

```

2.1. Árbol SLD

Consulta dibujada: `caminar(norte, norte, 7, [], P)`, que es el núcleo de `paseo(norte, Pasos)`: `paseo/2` solo añade `zona/1`, el conteo de puentes con `aggregate_all/3` (`Total = 7`) y `reverse/2`, predicados de biblioteca que no forman parte de la búsqueda.

El árbol completo tiene alrededor de 3 000 nodos y ninguna de sus ramas termina en éxito, que es la razón por la que `paseo(norte, Pasos)` responde `false`. Se dibujan los 8 primeros niveles:

- La primera cláusula de `caminar/5` falla de inmediato, porque `length([], 7)` es falso: aún no se ha cruzado ningún puente.
- La segunda cláusula prueba los puentes que tocan `norte` (`b1`, `b2` y `b5`). Las dos cláusulas de `cruce/3` permiten cruzarlos en ambos sentidos.
- En cada rama la negación `not(memberchk(...))` comprueba que el puente no se haya usado, y la búsqueda continúa con el segundo cruce desde `kneiphof` o desde `este`. Cuando una rama se queda sin puentes libres antes de usar los 7, falla, y Prolog retrocede a la siguiente alternativa.

Capítulo 3

Punto 2: Rutas más cortas

Listing 3.1: ProgIIIG1-Act02-Pto02-Andres_David_Rincon_Salazar.pl

```
/*
=====
Publication Date: 2026-10-02
Publication Time: 23:45
Code Version: 1.0
Author: Ing(c) Andres David Rincon Salazar
Programming Language: Prolog
Language Version: ISO/IEC 13211-1 (SWI-Prolog 10.0.2)
Compiler: SWI-Prolog 10.0.2
Operating System: Windows 11 - 24H2
Presented to: Ramiro Andres Barrios Valencia
University: Universidad Tecnologica de Pereira
Program: Ingenieria de Sistemas y Computacion

Program Description:
Shortest routes over the undirected road map of Spain (distances in km).
Roads are stored once as facts and made symmetric by a rule. Paths are
enumerated by backtracking with a list of visited cities (to avoid
infinite loops in the undirected graph); the shortest route is the
minimum over all of them. Validation queries are included at the end.

Predicates description:
- carretera(A, B, Km): (fact) road between A and B (stored once).
- conectadas(A, B, Km): (rule) symmetric closure of carretera/3.
- camino(Origen, Destino, Ciudades, Km): (rule) simple path, with its
  list of cities and total distance.
- ruta_corta(Origen, Destino, Ciudades, Km): (rule) minimum-distance path.
- hay_ruta(Origen, Destino): (rule) some path exists.

Warnings (Salvedades):
- The label of the Jaen-Granada road is rotated in the slide and could be
  read as 99 or as 66; 99 was used because it matches the real road
  distance between both cities.
- Ciudad Real, mentioned in the GPS article of the slides, is not in the
```

```

    map and therefore is not modeled.
- camino/4 enumerates all simple paths: exponential in general, but fine
  for this 15-city map.
=====
*/

%=====
% FACTS: roads of the map, carretera(CiudadA, CiudadB, Km)
%=====
carretera(coruna,      vigo,      171).
carretera(coruna,      valladolid, 455).
carretera(vigo,        valladolid, 356).
carretera(oviedo,      bilbao,    304).
carretera(valladolid, bilbao,    280).
carretera(valladolid, madrid,    193).
carretera(bilbao,      madrid,    395).
carretera(bilbao,      zaragoza, 324).
carretera(zaragoza,    madrid,    325).
carretera(zaragoza,    barcelona, 296).
carretera(barcelona,   gerona,    100).
carretera(barcelona,   valencia,  349).
carretera(madrid,      badajoz,   403).
carretera(madrid,      jaen,      335).
carretera(madrid,      albacete,  251).
carretera(albacete,    valencia,  191).
carretera(albacete,    murcia,    150).
carretera(valencia,    murcia,    241).
carretera(jaen,        sevilla,   242).
carretera(jaen,        granada,   99).
carretera(sevilla,     granada,   256).
carretera(sevilla,     cadiz,     125).
carretera(granada,     murcia,    278).

%=====
% RULE: roads are two-way
%=====
conectadas(A, B, Km) :- carretera(A, B, Km).
conectadas(A, B, Km) :- carretera(B, A, Km).

%=====
% RULES: simple paths by backtracking
% The accumulator Visitadas keeps cities already used so a path never
% revisits a city (otherwise A-B-A-B... would never end).
%=====
camino(Origen, Destino, Ciudades, Km) :-
    recorrer(Origen, Destino, [Origen], CiudadesInv, Km),
    reverse(CiudadesInv, Ciudades).

recorrer(Destino, Destino, Visitadas, Visitadas, 0).
recorrer(Actual, Destino, Visitadas, Ruta, Km) :-
    conectadas(Actual, Siguiente, Km1),
    \+ memberchk(Siguiente, Visitadas),

```

```

    recorrer(Siguiente, Destino, [Siguiente|Visitadas], Ruta, Km2),
    Km is Km1 + Km2.

%=====
% RULES: shortest route and reachability
%=====
ruta_corta(Origen, Destino, Ciudades, Km) :-
    aggregate_all(min(K, C), camino(Origen, Destino, C, K), min(Km, Ciudades)).

hay_ruta(Origen, Destino) :-
    camino(Origen, Destino, _, _), !.

/*
=====
Validation queries
=====
?- conectadas(madrid, valladolid, Km).
Km = 193.
?- conectadas(valladolid, madrid, Km).
Km = 193.
?- camino(coruna, vigo, C, Km).
C = [coruna, vigo],
Km = 171 ;
C = [coruna, valladolid, vigo],
Km = 811 ;
false.
?- ruta_corta(coruna, gerona, C, Km).
C = [coruna, valladolid, madrid, zaragoza, barcelona, gerona],
Km = 1369.
?- ruta_corta(cadiz, oviedo, C, Km).
C = [cadiz, sevilla, jaen, madrid, bilbao, oviedo],
Km = 1401.
?- ruta_corta(madrid, madrid, C, Km).
C = [madrid],
Km = 0.
?- hay_ruta(badajoz, gerona).
true.
?- hay_ruta(madrid, paris).
false.
=====
*/

```

3.1. Árbol SLD

Consulta dibujada: `recorrer(coruna, vigo, [coruna], R, Km)`, que es el núcleo de `camino(coruna, vigo, C, Km)`: `camino/4` solo invierte la lista resultante con `reverse/2`.

El árbol completo tiene más de 7 000 nodos: `recorrer/5` no se detiene al llegar al destino, sino que al hacer *backtracking* sigue explorando todos los caminos simples que salen de La

Coruña (601 en este mapa), aunque solo 2 terminan en Vigo. Esto es lo que hace exponencial a `ruta_corta/4`. Se dibujan los 10 primeros niveles:

- `conectadas/3` prueba primero `carretera(coruna, X, Km)` (Vigo y Valladolid) y luego el sentido inverso, que falla porque ninguna carretera termina en La Coruña.
- En la rama de Vigo, la primera cláusula de `recorrer/5` unifica `recorrer(vigo, vigo, ...)` y da la primera solución: `Km = 171`.
- Al pedir más soluciones, Prolog retrocede y continúa desde Vigo y desde Valladolid. La negación `not(memberchk(...))` corta las ramas que vuelven a una ciudad ya visitada.

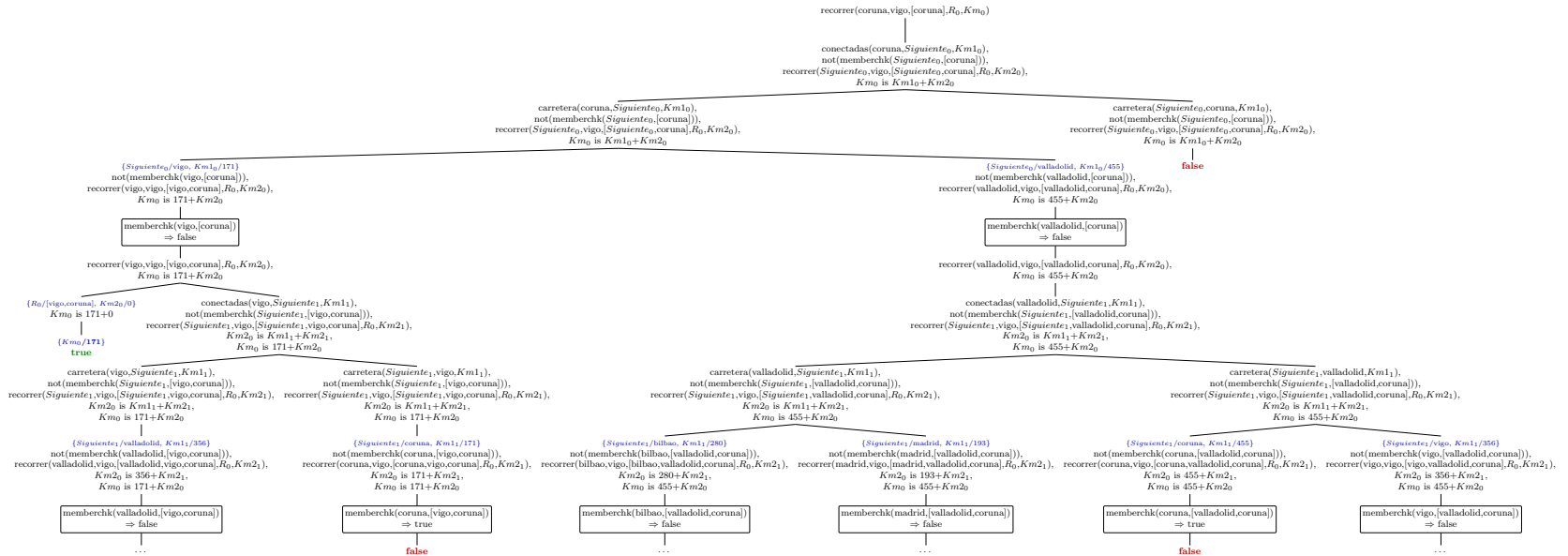


Figura 3.1: Árbol SLD de `recorrer(coruna, vigo, [coruna], R, Km)` (10 primeros niveles).

Capítulo 4

Punto 3: Grafo dirigido

Listing 4.1: ProgIIIG1-Act02-Pto03-Andres_David_Rincon_Salazar.pl

```
/*
=====
Publication Date: 2026-10-02
Publication Time: 23:45
Code Version: 1.0
Author: Ing(c) Andres David Rincon Salazar
Programming Language: Prolog
Language Version: ISO/IEC 13211-1 (SWI-Prolog 10.0.2)
Compiler: SWI-Prolog 10.0.2
Operating System: Windows 11 - 24H2
Presented to: Ramiro Andres Barrios Valencia
University: Universidad Tecnologica de Pereira
Program: Ingenieria de Sistemas y Computacion

Program Description:
Directed weighted graph of Canadian cities (Vancouver, Edmonton, Calgary,
Saskatoon, Regina, Winnipeg). Answers the questions of the slides:
- Is there a connection between Saskatoon and Vancouver?
- Which nodes is Regina connected to, and at what cost?
- Does a node have edges?
- What is the cost of going from X to Z passing through Y?
- Is it possible to travel from Edmonton to Calgary?
The last question needs recursion. A naive version (ruta_ingenua/2) is
included to show how SLD resolution loops on the cycle
edmonton -> saskatoon -> calgary -> edmonton (the first cycle reached with
the clause order of the facts), and a safe version with a list of visited
nodes (hay_camino/2) fixes it. Validation queries are
included at the end of the file.

Predicates description:
- arista(Origen, Destino, Costo): (fact) directed edge with its cost.
- conexion_directa(A, B, Costo): (rule) edge from A to B.
- conectado_con(Nodo, Otro, Costo, Sentido): (rule) neighbours of Nodo;
Sentido is salida (Nodo -> Otro) or entrada (Otro -> Nodo).
```

- *tiene_aristas(Nodo): (rule) Nodo has at least one incoming/outgoing edge.*
- *camino(Origen, Destino, Nodos, Costo): (rule) simple directed path.*
- *hay_camino(Origen, Destino): (rule) safe reachability.*
- *costo_via(X, Y, Z, Costo): (rule) cost of a path X -> Z that passes by Y.*
- *ruta_ingenua(Origen, Destino): (rule) naive reachability, may not end.*

Warnings (Salvedades):

- *Edges are directed: Vancouver reaches Saskatoon but not the other way.*
- *Costs are the numbers written on the edges of the slide.*
- *costo_via/4 considers simple paths X -> Z that contain Y as an intermediate node; there can be several answers (one per path).*
- *ruta_ingenua/2 is only a didactic example: it answers true for reachable pairs but, because of the cycle, keeps producing more true answers forever (one per lap) and, for unreachable pairs, keeps looping until it overflows the stack (about one minute with the default 1 GB stack limit of SWI-Prolog). Do not use it as a solution.*

```
=====
*/

%=====
% FACTS: directed edges, arista(Origen, Destino, Costo)
%=====
arista(vancouver, edmonton, 16).
arista(vancouver, calgary, 13).
arista(edmonton, saskatoon, 12).
arista(calgary, edmonton, 4).
arista(calgary, regina, 14).
arista(saskatoon, calgary, 9).
arista(saskatoon, winnipeg, 20).
arista(regina, saskatoon, 7).
arista(regina, winnipeg, 4).

%=====
% RULES: direct connections (questions 1 and 2 of the slides)
%=====
conexion_directa(A, B, Costo) :- arista(A, B, Costo).

conectado_con(Nodo, Otro, Costo, salida) :- arista(Nodo, Otro, Costo).
conectado_con(Nodo, Otro, Costo, entrada) :- arista(Otro, Nodo, Costo).

%=====
% RULE: a node has edges if some edge leaves or reaches it
%=====
tiene_aristas(Nodo) :-
    ( arista(Nodo, _, _) ; arista(_, Nodo, _) ), !.

%=====
% RULES: safe paths (visited list) -> Edmonton to Calgary question
%=====
camino(Origen, Destino, Nodos, Costo) :-
    recorrer(Origen, Destino, [Origen], NodosInv, Costo),
    reverse(NodosInv, Nodos).
```

```

recorrer(Destino, Destino, Visitados, Visitados, 0).
recorrer(Actual, Destino, Visitados, Ruta, Costo) :-
    arista(Actual, Siguiente, C1),
    \+ memberchk(Siguiente, Visitados),
    recorrer(Siguiente, Destino, [Siguiente|Visitados], Ruta, C2),
    Costo is C1 + C2.

hay_camino(Origen, Destino) :-
    camino(Origen, Destino, _, _), !.

%=====
% RULE: cost from X to Z passing through Y (Y is neither X nor Z)
%=====
costo_via(X, Y, Z, Costo) :-
    camino(X, Z, Nodos, Costo),
    Y \== X,
    Y \== Z,
    memberchk(Y, Nodos).

%=====
% DIDACTIC RULE: naive reachability (no visited list)
%
% SLD tree for ?- ruta_ingenua(edmonton, calgary).
%
%   ruta_ingenua(edmonton, calgary)
%   |-- clause 1: arista(edmonton, calgary, _)      FAIL
%   |-- clause 2: arista(edmonton, Z, _)  Z = saskatoon
%           ruta_ingenua(saskatoon, calgary)
%           |-- clause 1: arista(saskatoon, calgary, _)  SUCCESS (true)
%           |-- clause 2 (on backtracking): Z = calgary
%                   ruta_ingenua(calgary, calgary)
%                   |-- clause 1: FAIL
%                   |-- clause 2: Z = edmonton -> ruta_ingenua(edmonton, calgary)
%                           ... the initial goal again: INFINITE LOOP
%=====
ruta_ingenua(Origen, Destino) :- arista(Origen, Destino, _).
ruta_ingenua(Origen, Destino) :-
    arista(Origen, Intermedio, _),
    ruta_ingenua(Intermedio, Destino).

/*
=====
Validation queries
=====
% Question 1: connection between Saskatoon and Vancouver
?- conexion_directa(saskatoon, vancouver, C).
false.
?- hay_camino(saskatoon, vancouver).
false.
?- hay_camino(vancouver, saskatoon).
true.

```

```

% Question 2: nodes connected to Regina and cost of each connection
?- conectado_con(regina, Otro, Costo, Sentido).
Otro = saskatoon, Costo = 7, Sentido = salida ;
Otro = winnipeg, Costo = 4, Sentido = salida ;
Otro = calgary, Costo = 14, Sentido = entrada.

% Rule: node has edges
?- tiene_aristas(regina).
true.
?- tiene_aristas(toronto).
false.

% Rule: cost from X to Z passing through Y
?- costo_via(vancouver, calgary, winnipeg, C).
C = 55 ;
C = 49 ;
C = 54 ;
C = 31 ;
false.
?- costo_via(vancouver, regina, winnipeg, C).
C = 55 ;
C = 54 ;
C = 31 ;
false.
?- costo_via(winnipeg, regina, vancouver, C).
false.

% Most interesting question: Edmonton -> Calgary
?- hay_camino(edmonton, calgary).
true.
?- camino(edmonton, calgary, Nodos, Costo).
Nodos = [edmonton, saskatoon, calgary],
Costo = 21 ;
false.
?- ruta_ingenua(edmonton, calgary).
true ;
true ; % ... one more true per lap around the cycle: never ends (see SLD tree above)
?- ruta_ingenua(saskatoon, vancouver).
ERROR: Stack limit (1.0Gb) exceeded. % only after about one minute
=====
*/

```

4.1. Árboles SLD

4.1.1. Versión segura (árbol completo)

Consulta dibujada: `recorrer(edmonton, calgary, [edmonton], R, C)`, que es el núcleo de `camino(edmonton, calgary, Nodos, Costo)`. Este árbol se muestra completo

y es finito:

- Tiene una sola rama de éxito, con `R = [calgary, saskatoon, edmonton]` y `C = 21.camino/4` invierte la lista y responde `[edmonton, saskatoon, calgary]`.
- Al seguir buscando desde Calgary, la arista `calgary → edmonton` se poda porque `memberchk(edmonton, ...)` es verdadero: la lista de visitados corta el ciclo que hace infinita a la versión ingenua.
- Las demás ramas terminan en `false`: o vuelven a un nodo ya visitado, o llegan a Winnipeg, del que no sale ninguna arista.

4.1.2. Versión ingenua (árbol infinito)

Consulta dibujada: `ruta_ingenua(edmonton, calgary)`. Este árbol es infinito, así que se corta en el nivel 13, donde el objetivo inicial `ruta_ingenua(edmonton, calgary)` ya ha reaparecido dos veces en la rama izquierda (dos vueltas al ciclo `edmonton → saskatoon → calgary → edmonton`):

- Cada vuelta pasa por `ruta_ingenua(saskatoon, calgary)`, cuya primera cláusula tiene éxito por la arista directa. Por eso Prolog responde `true` una vez por vuelta, sin terminar nunca.
- Las ramas de la derecha (por Regina y Winnipeg) existen en el árbol, pero Prolog nunca llega a ellas: la búsqueda en profundidad se queda para siempre en la rama infinita de la izquierda. Por eso la estrategia de búsqueda de Prolog (en profundidad) no es completa.

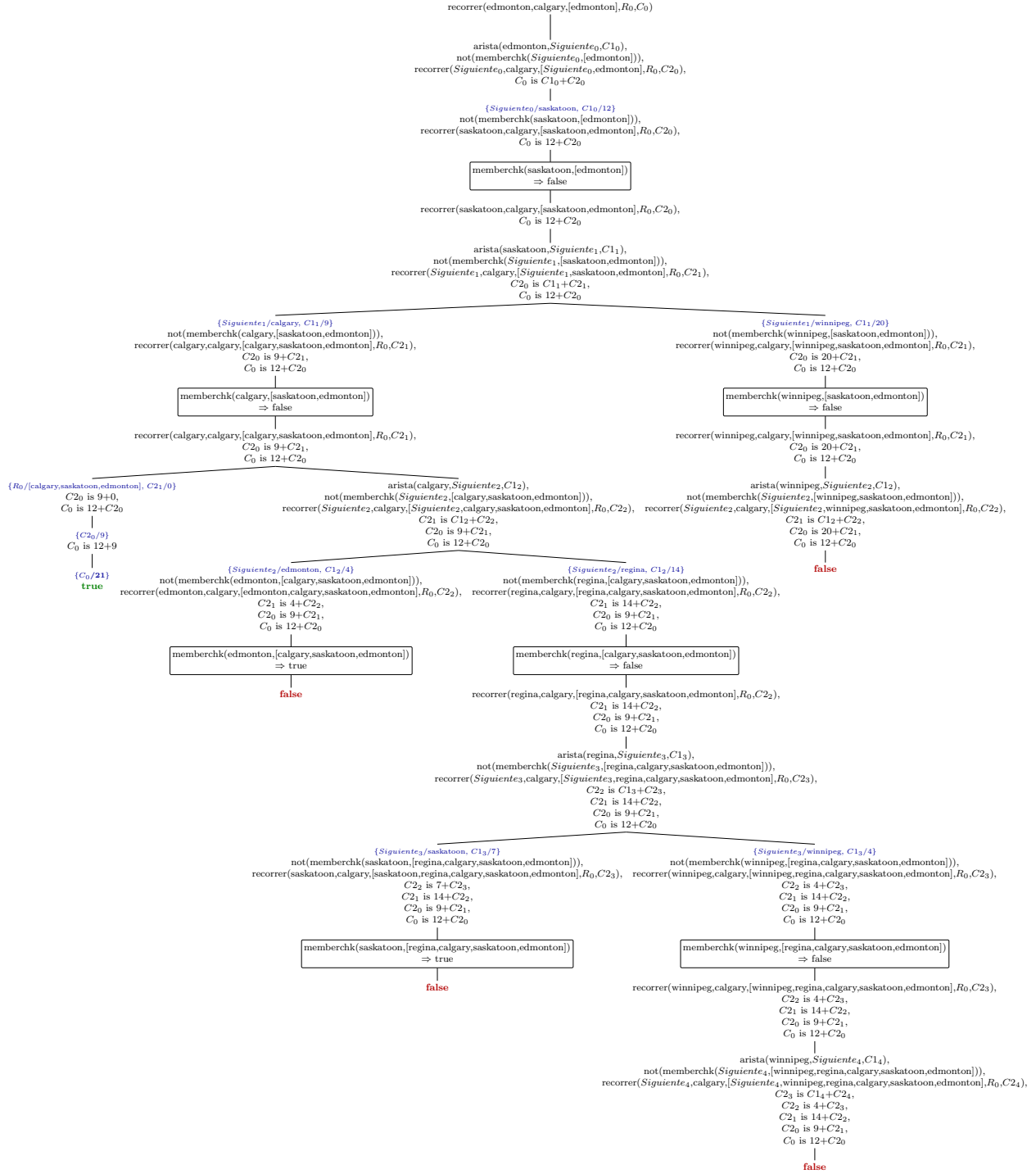


Figura 4.1: Árbol SLD completo de `recorrer(edmonton, calgary, [edmonton], R, C)`.

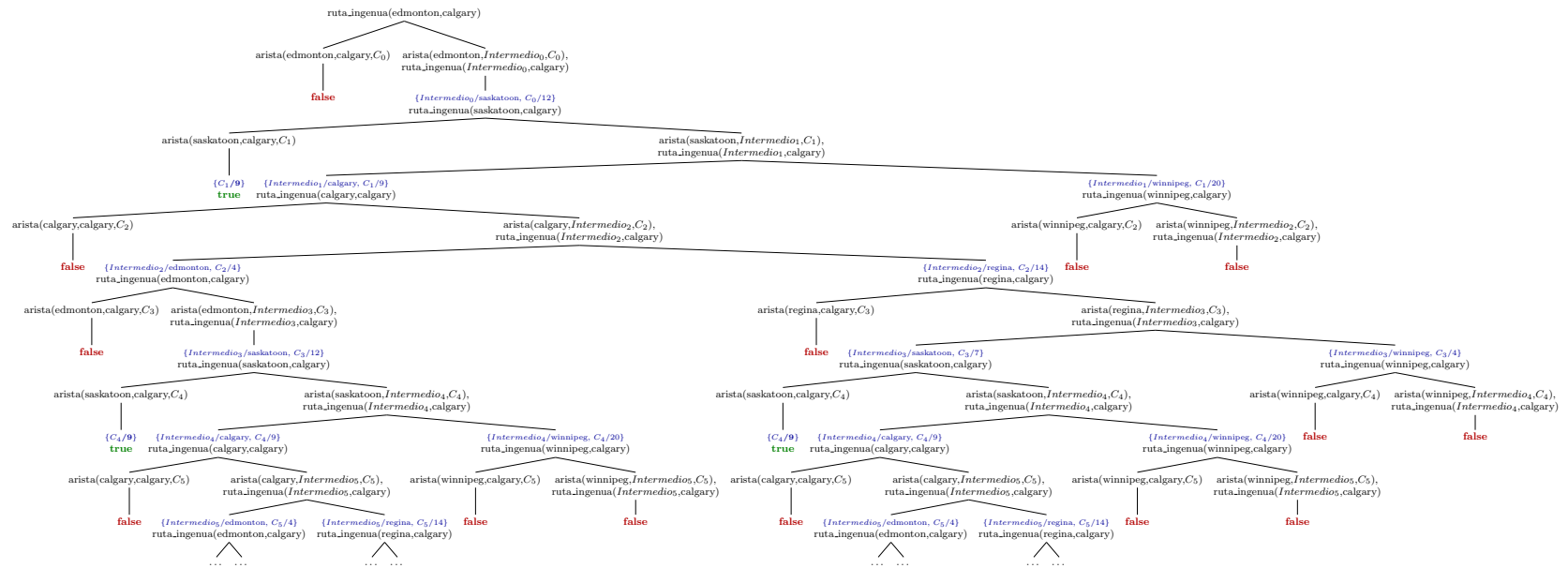


Figura 4.2: Árbol SLD de `ruta.ingenua(edmonton, calgary)` (13 primeros niveles de un árbol infinito).

Capítulo 5

Repositorio

El código fuente de esta actividad y el resto de actividades que ponga durante el semestre está y estarán disponible en GitHub:

`https://github.com/1594-adrs/ProgIIIG1-Andres\_David\_Rincon\_Salazar`